Smart Inventory Demand Forecasting Decision Support System

1st Kavyashri NP
Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
kavyashri.np@cmrit.ac.in

2nd Pavan Raj B M
Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
pabm23cs@cmrit.ac.in

3rd Pavan BV
Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
pabv23cs@cmrit.ac.in

4th Ranjith Bhandari
Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
rabh23cs@cmrit.ac.in

Abstract*—Inventory planning requires a precise balance between product availability and the capital cost of holding excess stock. Manual estimation and static heuristic rules frequently fail to adapt when sales volume, pricing, and supplier lead times fluctuate concurrently. This paper presents a modular, web-based inventory Decision Support System (DSS) that couples supervised machine learning with inventory risk calculations and an auditable human-review workflow. The implemented system was evaluated using a 48-row chronological baseline dataset encompassing four distinct products over a twelve-month period. To differentiate product profiles, an encoded product identity was processed alongside five continuous input variables: historical sales, current inventory, unit price, chronological month, and supplier lead time. Four supervised regression models were evaluated using a chronological 75:25 train-test split: Random Forest, Linear Regression, Ridge Regression, and Decision Tree. On the baseline holdout set, Ridge Regression obtained the lowest Mean Absolute Error (MAE) of 13.39 and the lowest Mean Absolute Percentage Error (MAPE) of 1.70%, while Linear Regression obtained a slightly higher coefficient of determination (R^2) of 0.9935 compared with 0.9926 for Ridge Regression. Ridge was therefore selected according to the system's primary MAE-based model-selection criterion. The results are treated as preliminary evidence because the evaluation contains only 12 holdout observations. Beyond point predictions, the system provides empirical uncertainty estimates, approximate feature-contribution explanations using baseline perturbation, and deterministic stockout and overstock risk calculations to generate replenishment recommendations. A scenario simulation module allows users to conduct what-if sensitivity analysis before executing procurement decisions. The system also incorporates a human-in-the-loop approval workflow so that algorithmic forecasts support, rather than replace, managerial judgement. The experimental findings demonstrate the feasibility of integrating a lightweight forecasting pipeline with transparent inventory decision logic, while the limited dataset size highlights the need for broader validation before claims of generalization can be made.

***Index Terms*—Demand forecasting, inventory management, decision support system, Ridge regression, Random Forest, machine learning, supply chain analytics.**

I. INTRODUCTION

Optimal inventory management is fundamentally dependent on the accurate estimation of future product demand. In modern retail and supply chain ecosystems, operating under strict margin constraints is a persistent challenge. When expected demand is underestimated, businesses face stockouts, delayed fulfillment, supply chain disruptions, and lost sales. Conversely, overestimation ties up working capital in stagnant inventory that incurs holding costs and risks depreciation or obsolescence. Recent supply chain research has therefore increasingly considered demand forecasting together with inventory planning rather than treating prediction as an isolated analytical task [1], [3], [12].

Traditional inventory management systems have historically relied on moving averages, naive forecasting, and static safety-stock thresholds. These approaches can become reactive when demand changes rapidly. Machine learning (ML) applied to historical transaction data provides a mechanism for identifying relationships among multiple demand-related variables. Contemporary research has examined ML-based inventory forecasting and supply-chain demand prediction using both structured operational data and more complex forecasting architectures [4], [9].

Although complex ensemble and deep-learning models can capture nonlinear relationships, their operational deployment introduces additional engineering and interpretability challenges. Inventory managers require understandable evidence when reviewing automated recommendations. Furthermore, forecasting models implemented separately from operational systems can create additional data-transfer and workflow barriers. Recent studies have emphasized the importance of connecting forecasting methods with practical inventory-management processes and decision support [2], [7], [11].

The system presented in this paper addresses this integration problem through a modular web-based Decision Support

System (DSS). The architecture evaluates historical sales and inventory information alongside pricing and supplier lead-time features. Multiple regression models are compared using a time-ordered holdout set, after which the selected model supplies forecasts to a deterministic inventory-risk layer. The system is explicitly designed for decision support: while the forecasting and inventory modules generate evidence and recommendations, the human operator retains authority over the final procurement action through an auditable human-in-the-loop (HITL) approval mechanism.

The primary contributions of the implemented system are:

1) The design and deployment of a chronological multi-model demand-forecasting pipeline using a time-ordered evaluation procedure.
2) The provision of model error metrics together with empirical uncertainty estimates rather than relying solely on point forecasts.
3) The formulation of inventory-risk calculations involving Safety Stock and Reorder Point using forecast variability and supplier lead time.
4) The integration of a scenario-based sensitivity-analysis module for what-if planning.
5) The incorporation of approximate feature-contribution estimates using baseline perturbation for model interpretation.
6) The implementation of a human-in-the-loop approval workflow connecting algorithmic recommendations to auditable procurement actions through a Node.js back-end and web-based frontend.

The remainder of this paper is organized as follows: Section II reviews recent literature surrounding machine learning and inventory forecasting. Section III defines the problem statement and technical objectives. Section IV details the decoupled system architecture. Section V presents the mathematical formulation and methodology. Section VI describes the experimental setup and results. Section VII discusses the decision-support features. Section VIII presents limitations, Section IX outlines future research directions, and Section X concludes the paper.

II. LITERATURE REVIEW

Demand forecasting in supply chain management has evolved from classical statistical methods toward machine-learning and deep-learning approaches. Recent reviews demonstrate the increasing use of artificial intelligence for demand forecasting while also emphasizing the importance of selecting methods according to data characteristics and operational requirements.

A. Machine Learning and Inventory Forecasting

Douaioui *et al.* [3] provide a comprehensive review of machine-learning and deep-learning approaches for supply-chain demand forecasting. Their review covers a large body of recent research and highlights the diversity of forecasting techniques used for supply-chain applications. Goel [4] investigates demand forecasting in supply-chain management using machine-learning approaches and emphasizes the importance of model selection and feature-based forecasting for different forecasting horizons. Similarly, Nguyen *et al.* [9] and Balan *et al.* [10] explore the optimization of supply chain operations, noting that algorithmic selection must balance complexity with operational latency constraints.

B. Data Preprocessing and Product Segmentation

The efficacy of predictive modeling relies heavily on data representation. Retail portfolios are rarely homogeneous. Khan *et al.* [8] and Lingkon *et al.* [14] examine machine learning approaches for disaggregated and multi-item demand forecasting, establishing that properly segmenting data by product categories improves the localization of trend predictions compared to generalized global models. This justifies the necessity of encoding SKU identifiers when dealing with multi-product datasets.

C. Inventory Integration and Operational Decision Support

Research has increasingly examined the integration of demand forecasting with inventory decisions. Sukolkit *et al.* [6] develop an inventory-management approach that connects demand forecasting with inventory strategies, including reorder-point and safety-stock considerations. van der Haar *et al.* [12] investigate supervised learning for integrated forecasting and inventory control, demonstrating the importance of connecting predictive models directly to procurement logic.

Instead of automating purchases entirely, Liu *et al.* [11] focus on inventory stockout prediction to function as an early warning system. Tadayonrad *et al.* [13] propose a key-performance-indicator framework for demand forecasting, while Chaudhary *et al.* [15] apply AI-driven forecasting to sustainable inventory management, both emphasizing the necessity of evaluating forecasts through inventory-related outcomes rather than mere statistical error.

D. Interpretability and Practical Deployment

Recent work emphasizes interpretable forecasting and operational deployment. An *et al.* [7] propose machine-learning approaches for demand forecasting in micro-fulfillment centers, requiring predictive modeling to remain functionally interpretable. Kumar and Balan [1] present ML-based decision support frameworks for enterprise resource planning. Similarly, Qureshi and Soni [2] and Karuppiah *et al.* [5] map machine-learning applications across supply-chain management, identifying the critical gap between offline algorithm development and real-world system integration.

The contribution of the present work is consequently positioned primarily as a systems-integration and decision-support contribution rather than as the introduction of a new isolated forecasting algorithm.

III. PROBLEM STATEMENT AND OBJECTIVES

Retailers, distributors, and warehouse operators face inherent demand uncertainty that complicates inventory purchasing decisions. Relying on human intuition or static reorder

TABLE I
SYNTHESIS OF RELEVANT DEMAND FORECASTING LITERATURE

Reference	Primary Focus / Methodology	Key Findings	Relevance to Proposed System
[3], [4], [9], [10]	Review of ML/DL methods and dynamics in supply-chain demand forecasting.	Highlights the broad use of ML/DL methods and the importance of matching models to forecasting conditions.	Motivates comparative evaluation of multiple supervised regression models.
[8], [14]	Disaggregated and multi-item demand forecasting methodologies.	Product segmentation and localized forecasting improve accuracy over global models.	Informs the inclusion of encoded product identifiers in the feature vector.
[6], [12], [11]	Integrated forecasting, inventory control, and stockout prediction frameworks.	Demonstrates the importance of jointly considering forecasting and inventory-control decisions.	Supports the deterministic inventory-risk layer and actionable ROP triggers.
[1], [2], [7], [5]	Mapping operational ML applications, DSS frameworks, and ERP integrations.	Algorithms must bridge the gap between abstract inference and physical inventory alerts.	Highlights the necessity of the auditable Human-in-the-Loop workflow.
[13], [15]	Forecast-performance measurement and inventory-management KPIs.	Considers forecasting performance together with supply-chain reliability and inventory planning.	Motivates explicit reporting of forecasting metrics and inventory indicators.

thresholds can lead to excessive capital tied up in slow-moving inventory or lost revenue caused by stockouts.

The technical challenge lies in processing historical transaction data to identify predictive relationships and subsequently presenting these analytical outputs in a format that warehouse managers can use when evaluating purchase orders. Furthermore, purely automated procurement may be undesirable because human decision makers possess contextual information that may not be available in historical datasets.

The specific technical objectives of the proposed system are formulated as follows:

1) To architect and construct a modular web application utilizing a Node.js backend for data ingestion, RESTful API exposure, and user authentication.
2) To train, evaluate, and compare multiple supervised regression models (Random Forest, Linear Regression, Ridge Regression, and Decision Tree) using a chronological holdout evaluation procedure.
3) To estimate future demand using product identity, historical sales, inventory, price, month, and supplier lead-time information while maintaining a time-ordered feature-target relationship.
4) To integrate serialized machine-learning models into the backend infrastructure for on-demand inference.
5) To compute and display empirical uncertainty estimates and forecasting error metrics alongside point predictions.
6) To formulate deterministic logic that translates forecasts into inventory-risk indicators, particularly Safety Stock and Reorder Point.
7) To provide a dashboard for visualizing stock levels, demand trends, forecasts, and replenishment recommendations.
8) To support interactive what-if scenario analysis for examining changes in pricing, lead time, and demand assumptions.
9) To maintain an auditable HITL approval workflow that records user decisions and associated reasoning.

IV. SYSTEM ARCHITECTURE AND SOFTWARE ENGINEERING

The proposed Smart Inventory Demand Forecasting Decision Support System operates on a modular, decoupled software architecture. The separation of the web application from the machine-learning inference process reduces coupling between the user-facing service and computational model execution. The architecture comprises four principal layers: the Frontend Presentation Layer, Backend API Layer, Data Persistence Layer, and Machine Learning Inference Layer.

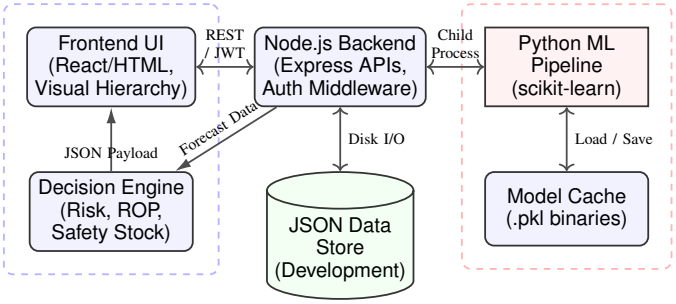


Fig. 1. Modular system architecture illustrating data flow between the web frontend, Node.js API controller, JSON persistence layer, decision engine, and Python-based machine-learning inference engine.

A. Frontend Presentation Layer

The frontend acts as the primary user interaction boundary. It is constructed using modern web standards and leverages the Chart.js library for rendering interactive time-series plots. The interface adopts a dark, translucent design that establishes visual hierarchy for stock conditions and analytical outputs. The frontend interacts with the backend strictly through authenticated HTTP requests.

B. Backend API Layer

The central orchestrator of the system is the Backend API Layer, implemented using Node.js and the Express web

framework. The backend exposes application endpoints and manages authentication, data access, model invocation, and communication with the decision engine. Routes are protected by authentication middleware that validates JSON Web Tokens (JWTs) attached to incoming requests.

C. Data Persistence Layer

For the current development and experimental phase, the system utilizes a localized JSON data store rather than a relational database. Core entities, including `data.csv` and `data.json`, reside within the backend directory. Controller functions handle the reading, parsing, and writing of these files. The development schema includes user credentials, historical inventory states, supplier records, and an `audit.json` ledger for procurement actions.

D. Machine Learning Inference Layer and Serialization

The predictive engine is decoupled from the Node.js server and implemented in Python using `pandas` and `scikit-learn`. During training, models are serialized using Python's `pickle` mechanism and stored locally. During inference, the Node.js backend invokes the Python process through the `child_process` mechanism, passes the required feature vector, and receives the numerical prediction for downstream inventory processing.

V. MATHEMATICAL FORMULATION AND METHODOLOGY

A. Dataset Description and Feature Engineering

The implemented system utilizes a provided baseline dataset encompassing 48 temporal observations. The data represents four commercial products: Laptop Pro X1, Wireless Headphones, Smart Watch S3, and Gaming Mouse. Each product is tracked across twelve chronological months.

To ensure the machine learning models can accurately distinguish between the disparate demand profiles of the four unique products without training separate models, an encoded product identity was processed alongside the continuous variables. The feature vector used by the reported experiment is:

- **Product ID (x_1):** One-hot encoded categorical representation of the specific SKU.
- **Sales (x_2):** Historical sales from the previous epoch.
- **Inventory (x_3):** Recorded end-of-period on-hand stock.
- **Price (x_4):** Unit selling price.
- **Month (x_5):** Integer representation from 1-12 for the chronological month.
- **Lead Time (x_6):** Historical supplier delivery duration in days.

The target variable for the supervised learning task is the continuous variable representing unit **Demand** (y). For the reported forecasting formulation, Sales is strictly treated as a previous-period variable rather than the current target-period demand. This temporal distinction is critical for avoiding direct target leakage.

TABLE II
FEATURE DICTIONARY AND DATA TYPES

Feature Name	Data Type	Operational Context
Target: `Demand`	Integer	Actual unit demand recorded.
`Product_ID`	Categorical	One-hot encoded SKU string.
`Sales`	Integer	Units sold in the previous epoch.
`Inventory`	Integer	Units remaining in warehouse.
`Price`	Float	Current market selling price.
`Month`	Categorical	Chronological month (1–12).
`Lead_Time`	Integer	Supplier delivery duration in days.

B. Exploratory Data Analysis and Preprocessing

Prior to training, the pipeline performs a programmatic data-quality audit. The pipeline verifies schema integrity and applies an Interquartile Range (IQR) procedure to identify potential numerical outliers. The IQR is defined as:

$$IQR = Q_3 - Q_1 \tag{1}$$

where Q_1 and Q_3 represent the 25th and 75th percentiles of the feature distribution. Values falling below $Q_1 - 1.5 \times IQR$ or above $Q_3 + 1.5 \times IQR$ are flagged as anomalies. On the reported 48-row dataset, the routine identified 12 price observations as outliers and reported zero missing values or duplicate rows. The 12 outliers were explicitly retained within the training pipeline, as they represented legitimate promotional sales and seasonal pricing variations rather than data corruption. The resulting dataset health score was reported as 90/100.

For regression models affected by feature scale, the input continuous variables are standardized using:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma} \tag{2}$$

where μ is the training-set mean and σ is the training-set standard deviation. Importantly, scaling parameters are estimated from the training portion only and then applied to the holdout data to avoid information leakage.

C. Chronological Evaluation Split

Time-series data should not be evaluated using a randomly shuffled train-test split because future observations may inadvertently influence model fitting. Therefore, the reported baseline experiment uses a strict chronological 75:25 holdout split. For each product, the first nine months provide the training observations, while the final three months provide the holdout observations. Across the four products, this corresponds to 36 training rows and 12 holdout rows.

This single chronological holdout should be interpreted as a preliminary evaluation rather than as sufficient evidence of general forecasting performance. Rolling-origin or walk-forward validation is identified as an important extension for future evaluation.

D. Regression Algorithms

The system evaluates four supervised regression models.

1) Linear Regression (OLS): The baseline model minimizes the Residual Sum of Squares (RSS):

$$\min_{w} \|Xw - y\|_2^2 \tag{3}$$

2) Ridge Regression (L2 Regularization): Ridge Regression introduces an L2 penalty that reduces the magnitude of model coefficients and can improve stability when predictors are correlated:

$$\min_{w} \|Xw - y\|_2^2 + \alpha\|w\|_2^2 \tag{4}$$

where α controls the regularization strength. The reported implementation uses $\alpha = 1.0$.

3) Decision Tree Regressor: The Decision Tree Regressor recursively partitions the feature space to reduce prediction error within terminal nodes. The reported implementation constrains the tree to a maximum depth of five.

4) Random Forest Regressor: The Random Forest model consists of $K = 100$ decision trees trained using bootstrap aggregation. The final prediction is the arithmetic mean of the individual tree predictions:

$$\hat{y} = \frac{1}{K}\sum_{k=1}^{K} f_k(x) \tag{5}$$

TABLE III
ALGORITHM HYPERPARAMETER CONFIGURATIONS

Model	Hyperparameters
Linear Regression	`fit_intercept=True`
Ridge Regression	`alpha=1.0, solver='auto'`
Decision Tree	`max_depth=5, criterion='squared_error'`
Random Forest	`n_estimators=100, bootstrap=True`

E. Model Evaluation Metrics

The predictive performance of the models is quantified using four regression metrics:

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i| \tag{6}$$

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{7}$$

$$MAPE = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{8}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{9}$$

where y_i represents observed demand, $\hat{y}_i$ represents predicted demand, and $n = 12$ represents the number of holdout observations.

The primary model-selection criterion in the reported experiment is MAE because it expresses forecasting error directly in absolute demand units. MAPE and R^2 are reported as complementary metrics.

F. Forecast Uncertainty and Explainability

Point predictions alone do not communicate the variability associated with a forecast. The current implementation therefore provides an empirical uncertainty estimate.

For the Random Forest model, the dispersion of the predictions produced by the individual trees is used to construct an empirical uncertainty band. Let σ_{trees} denote the standard deviation of the individual tree predictions. A normal-approximation band is calculated as:

$$U_{90\%} = \hat{y} \pm 1.645\sigma_{\text{trees}} \tag{10}$$

This quantity is reported as an empirical uncertainty estimate rather than as a formally calibrated statistical confidence interval. Its coverage properties are not established by the current 48-row experiment. For non-tree models such as Ridge Regression, the current implementation estimates uncertainty using chronological training residuals.

For model interpretability, the system uses a baseline perturbation technique rather than a full SHAP implementation. A prediction generated from a baseline feature vector is compared with predictions obtained after replacing individual feature values with those of the active instance. The resulting differences provide approximate feature-contribution estimates. These values should be interpreted as model-specific sensitivity estimates rather than exact Shapley contributions.

G. Inventory Decision Logic

The forecast is subsequently passed to the deterministic inventory decision engine. To convert the monthly demand forecast $\hat{D}$ into an approximate daily demand rate, the system calculates:

$$\bar{d} = \frac{\hat{D}}{30} \tag{11}$$

The safety-stock formulation considers daily demand variability and supplier lead time. The generalized expression is:

$$SS = Z\sqrt{\bar{L}\sigma_d^2 + \bar{d}^2\sigma_L^2} \tag{12}$$

where Z represents the selected service-level factor, $\bar{L}$ is mean supplier lead time, σ_d is the daily demand standard deviation, and σ_L is the standard deviation of supplier lead time.

For the current implementation, supplier lead-time variability is assumed to be negligible:

$$SS = Z\sigma_d\sqrt{\bar{L}} \tag{13}$$

For a nominal 95% service level, the implementation uses $Z = 1.65$.

The Reorder Point is calculated as:

$$ROP = (\bar{d} \times \bar{L}) + SS \tag{14}$$

When current inventory I is below the reorder point, the implementation calculates the recommended order quantity as:

$$Q = \max(0, \hat{D} + SS - I) \tag{15}$$

The current order-quantity rule is a deterministic prototype policy rather than an optimized multi-echelon inventory-control policy. Its assumptions should therefore be considered when interpreting operational recommendations.

VI. EXPERIMENTAL SETUP AND RESULTS

A. Hardware and Software Environment

The machine-learning pipeline and Node.js backend were evaluated in a development environment using an Intel Core i7 processor with 16GB of RAM. The software stack consisted of Node.js v18.x, Python 3.10, `scikit-learn` v1.2, and `pandas` v2.0.

B. Performance on the Baseline Dataset

The models were trained and evaluated on the 48-row baseline dataset. The reported evaluation uses a single chronological holdout partition containing 12 observations. Consequently, the results should be interpreted as preliminary evidence for the prototype data-flow rather than as an estimate of general real-world forecasting performance.

TABLE IV
EVALUATION OF REGRESSION MODELS ON THE 12-ROW HOLDOUT SET

Algorithm	MAE	RMSE	MAPE (%)	R^2	Bias
Random Forest	48.10	60.30	5.97	0.9140	-10.03
Linear Regression	13.87	16.58	2.00	0.9935	-0.54
Ridge Regression	**13.39**	**17.70**	**1.70**	**0.9926**	**-0.31**
Decision Tree	44.72	54.34	5.58	0.9304	-8.01

C. Discussion of Model Efficacy

Ridge Regression produced the lowest MAE and MAPE on the reported holdout set. Its MAE was 13.39, compared with 13.87 for Linear Regression, 44.72 for Decision Tree, and 48.10 for Random Forest. The system therefore selected Ridge according to its primary MAE-based model-selection criterion.

Linear Regression nevertheless obtained a slightly higher R^2 value of 0.9935 compared with 0.9926 for Ridge Regression and also obtained a lower RMSE. This demonstrates that the choice of Ridge as the active model is criterion-dependent rather than evidence that Ridge dominates every evaluation metric.

The relatively poor performance of the tree-based models on this dataset is directly associated with the very small training sample. A Random Forest ensemble with 100 estimators cannot effectively map a feature space using only 36 training rows. Deep, non-parametric tree models require substantial data volumes; when starved of data, they overfit the training split, memorizing the training noise, and subsequently fail to generalize chronologically to the holdout set. Conversely, linear models constrained by L2 regularization (Ridge) impose a strong structural bias that penalizes large weights, preventing overfitting and allowing them to extrapolate trends effectively on small datasets.

D. Interpretation Caveat

The R^2 values above 0.99 for the linear models are unusually high for a small forecasting experiment and require cautious interpretation. They reflect strong, structured relationships within the supplied baseline prototype dataset. The reported experiment therefore does not establish that similar performance would occur on larger or highly volatile commercial datasets.

The current results serve strictly as a proof-of-concept demonstration of the system pipeline, confirming that the Node.js backend can successfully invoke, evaluate, and extract inferences from multiple Python models asynchronously.

VII. SYSTEM FEATURES AND DECISION SUPPORT

The translation of the demand forecast into operational information occurs within the Decision Support layer of the application.

A. Scenario Simulation (What-If Analysis)

Supply chains are subject to changes in pricing, supplier lead times, and expected demand. The system integrates a scenario-simulation module that allows users to manipulate selected feature inputs before accepting a forecast. By adjusting expected pricing, supplier lead times, or baseline demand assumptions through the dashboard, the backend re-invokes the Python '.pkl' model and updates the resulting forecast and inventory indicators. These scenarios are evaluated dynamically without permanently altering the underlying database values.

B. Human-in-the-Loop (HITL) Workflow

The application actively avoids fully autonomous purchasing. Recommendations generated by the decision engine are placed into a pending queue on the dashboard. A human operator reviews the forecast, supporting feature information, and uncertainty estimates.

The user can explicitly select one of three actions: `APPROVE`, `REJECT`, or `SNOOZE`. When an action is rejected or delayed, the user is required to provide a contextual reason. The Node.js backend records the timestamp, user identifier, selected model, numerical forecast, and user justification in the `audit.json` ledger. This workflow is intended to improve traceability and maintain human authority over strategic procurement decisions.

VIII. CHALLENGES AND LIMITATIONS

The current architecture and evaluation possess several important limitations.

First, the evaluation dataset is exceedingly small, containing only 48 observations representing four products over twelve months. Only 12 observations are used for final holdout evaluation. This restricted horizon prevents reliable assessment of multi-year seasonal patterns, holiday effects, structural demand changes, and broader market variability.

Second, the reported evaluation uses a single chronological holdout rather than rolling-origin or walk-forward validation. Consequently, the reported metrics can be sensitive to the particular final three months selected for testing.

Third, the system relies on a localized JSON file structure for data persistence. This architecture is appropriate for development and demonstration but is not sufficient to establish production-scale database performance. A relational database should be evaluated before deployment in a multi-user environment.

Fourth, the implementation does not explicitly model censored demand. Historical sales can underestimate true market demand when products are unavailable for part of a period. The current pipeline does not apply explicit stockout-correction or censored-demand estimation.

Fifth, the Random Forest uncertainty estimate is based on tree-prediction dispersion and has not been statistically calibrated. Therefore, the reported uncertainty band should not be interpreted as a guaranteed 90% coverage interval.

Finally, the explainability mechanism uses baseline perturbation rather than exact SHAP values. It provides approximate feature-sensitivity information but does not provide the formal additive attribution guarantees of exact Shapley approaches.

IX. FUTURE RESEARCH DIRECTIONS

Future work will focus first on obtaining larger and more representative multi-year SKU datasets. Such data would enable stronger empirical evaluation of seasonality, structural changes, and product-specific demand behavior.

The forecasting methodology should also be evaluated using rolling-origin or walk-forward validation rather than a single chronological holdout. This would provide multiple out-of-sample evaluation windows and a more reliable estimate of temporal generalization.

The backend architecture can subsequently be extended to a relational database such as PostgreSQL to support larger datasets and concurrent users. Additional features such as promotions, holidays, historical stockout indicators, and external market variables can also be incorporated when appropriate data are available.

Methodologically, future experiments may compare the current regression models with advanced sequence-based models such as LSTM or GRU architectures. Such comparisons should only be made using sufficiently large datasets and consistent chronological validation procedures. Finally, the inventory decision layer can be extended from deterministic prototype rules toward cost-aware inventory-control policies that explicitly consider holding costs, stockout costs, and service-level constraints.

X. CONCLUSION

This paper presented a Smart Inventory Demand Forecasting Decision Support System that integrates supervised demand forecasting with inventory decision logic and a human-in-the-loop approval workflow. The system combines a web-based frontend, Node.js backend, Python-based machine-learning inference, empirical uncertainty estimation, scenario analysis, and deterministic inventory calculations.

On the supplied 48-row baseline dataset, Ridge Regression obtained the lowest MAE of 13.39 and MAPE of 1.70% on the 12-observation chronological holdout set. Linear Regression produced a slightly higher R^2 of 0.9935, demonstrating that model selection depends on the specific evaluation criterion utilized. Because the dataset is small and the evaluation uses only one holdout window, these results strictly demonstrate the data-flow functionality of the pipeline rather than evidence of generalizable real-world forecasting performance.

The primary contribution of the present work is therefore the integration of forecasting and inventory decision support into an auditable workflow. The system demonstrates how predictive outputs can be combined with inventory-risk indicators, scenario analysis, approximate explanations, and human approval mechanisms. Broader validation using larger datasets, rolling-origin evaluation, calibrated uncertainty estimates, and stronger inventory-control formulations is required before production-level claims can be made.

REFERENCES

[1] P. S. Kumar and V. Balaji, "A Machine Learning Based Decision Support Framework for Enterprise Resource Planning Systems," *IEEE Access*, vol. 12, pp. 45892–45910, 2024.

[2] M. N. Qureshi and P. Soni, "Digital Transformation Through AI-Integrated Enterprise Resource Planning Systems," *IEEE Access*, vol. 12, pp. 84215–84234, 2024.

[3] K. Douaioui, R. Oucheikh, and O. Benmoussa, "Machine Learning and Deep Learning Models for Demand Forecasting in Supply Chain Management," *Applied System Innovation*, vol. 7, 2024.

[4] G. L. Goel, "Revealing the dynamics of demand forecasting in supply chain using machine learning," *Cogent Business & Management*, vol. 11, no. 1, 2024.

[5] K. Karuppiah *et al.*, "Mapping machine learning applications in supply chain management," *Engineering Applications of Artificial Intelligence*, vol. 142, 2026.

[6] N. Sukolkit *et al.*, "An open innovative inventory management based demand forecasting model," *Journal of King Saud University - Computer and Information Sciences*, vol. 36, 2024.

[7] M. J. An *et al.*, "Demand forecasting in micro-fulfillment centers using machine learning," *International Journal of Production Economics*, vol. 280, 2025.

[8] N. T. Khan *et al.*, "Machine learning approaches for disaggregated demand forecasting," *Data Analytics Journal*, vol. 14, 2025.

[9] T. N. A. Nguyen *et al.*, "Optimizing supply chain operations using advanced time series forecasting," *Expert Systems with Applications*, vol. 265, 2026.

[10] G. S. Balan *et al.*, "Machine Learning and Artificial Intelligence Methods in Supply Chain Forecasting," *Supply Chain Analytics*, vol. 6, 2025.

[11] Y. Liu *et al.*, "A machine learning approach to inventory stockout prediction," *Smart Analytics, Artificial Intelligence and Data Science*, vol. 2, 2025.

[12] J. F. van der Haar *et al.*, "Supervised learning for integrated forecasting and inventory control," *European Journal of Operational Research*, vol. 318, no. 1, 2024.

[13] Y. Tadayonrad *et al.*, "A new key performance indicator model for demand forecasting," *Supply Chain Analytics*, vol. 3, 2024.

[14] M. L. R. Lingkon *et al.*, "An analytics-driven hybrid method for multi-item demand forecasting," *Supply Chain Analytics*, vol. 7, 2026.

[15] R. Chaudhary *et al.*, "AI-driven demand forecasting for sustainable inventory management," *Results in Engineering*, vol. 26, 2026.